
Labor-/Projektbericht • Abgabe am Ende des Semesters, Umfang: ca. 15-20 Seiten • Inhalt / Strukturierung (kann an Struktur eines Pflichtenhefts angelehnt sein)

1. ☐ Einleitung
2. ☐ Motivation, Zielsetzung
3. ☐ Detaillierte Problemstellung
4. ☐ Technologieauswahl (mit Begründung)
5. ☐ Use Cases
6. ☐ Muss-/Kann-Kriterien
7. ☐ Umsetzung / Implementierung
 1. Frontend (React):
 1. ☐ Designprozess (Figma)
 2. ☐ Komponenten (Kalender Komponente...)
 3. ☐ Requests zum Backend Server
 2. Backend (.NET API):
 1. ☐ API Endpunkte
 2. ☐ Kalenderdaten-Verarbeitung
 3. ☐ Gruppenverwaltung
 4. ☐ Datenbank (PostgreSQL): Datenbankstruktur
 5. ☐ Kommunikation zu anderen Systemen (Google Auth, RaumZeit API)
 3. Deployment
 1. ☐ In welcher Umgebung wurde es deployed
 2. ☐ Containerisierung (neues Feature von dotnet erwähnen?)
8. ☐ Fazit
 - ☐ Was lief gut / wo gab es Herausforderungen?
 - ☐ Ausblick (siehe Leitfragen, was sind Vorteile & was sind Schwächen des aktuellen Architektur-Designs?)
9. ☐ Literaturverzeichnis (+ Verweise wo AI eingesetzt wurde)

Lernzeit

Grundlegende Problemstellung

Studierende stehen regelmäßig vor der Herausforderung, gemeinsame Termine für Lern- oder Projektgruppen zu koordinieren. Aufgrund individueller und häufig stark unterschiedlicher Stundenpläne gestaltet sich die Abstimmung geeigneter Zeitfenster als zeitaufwendig und ineffizient.

In der Praxis erfolgt die Terminfindung meist über Gruppenchats oder persönliche Absprachen. Dabei werden Vorschläge von einzelnen Gruppenmitgliedern eingebracht und anschließend von den übrigen Teilnehmenden auf mögliche Konflikte geprüft. Dieser iterative Abstimmungsprozess führt häufig zu einem „Hin und Her“-Austausch, der sowohl zeitintensiv als auch organisatorisch aufwendig ist.

Die Anwendung **Lernzeit** adressiert dieses Problem, indem sie die Terminfindung automatisiert unterstützt. Nutzerinnen und Nutzer können Gruppen erstellen und ihre individuell blockierten Zeiten hinterlegen. Auf Basis dieser Informationen generiert die Anwendung geeignete Terminvorschläge, zu denen alle Gruppenmitglieder verfügbar sind. Ziel ist es, den Abstimmungsaufwand zu reduzieren und eine effiziente sowie transparente Terminplanung zu ermöglichen.

Architekturvorschlag

Es wurden im Rahmen des Pflichtenhefts verschiedene Systemarchitekturen verglichen. Eine Gegenüberstellung hierzu ist in Tabelle 1 zu finden.

Es wurde sich im Rahmen des Projekts für eine Client-Server Architektur entschieden. Die erhöhte Komplexität alternativer Architekturstile steht in keinem angemessenen Verhältnis zu deren potenziellen Vorteilen für das Lernzeitprojekt im Rahmen des Labors.

Durch eine konsequente modulare Trennung der Funktionalitäten wird jedoch sichergestellt, dass eine spätere Migration zu einer Microservice-Architektur grundsätzlich möglich ist.

Architektur	Pro	Kontra
Client-Server	• Zentrale Kontrolle und Verwaltung • Einfache Implementierung • Gute Sicherheit durch Backend • Anonymisierung zentral möglich	• Single Point of Failure • Skalierungsprobleme bei hoher Last • Kann ggf. zu unwartbarem Monolithen wachsen
Microservice Architektur	• Gute Skalierbarkeit • Klare Trennung der Verantwortlichkeiten • Services unabhängig entwickelbar • Flexibel erweiterbar	• Höhere Komplexität • Aufwendiges Deployment • Kommunikation zwischen Services notwendig • Fehler schwerer zu debuggen
Peer-to-Peer	• Keine zentrale Instanz nötig • Gute Skalierbarkeit • Geringe Serverlast • Hohe Ausfallsicherheit	• Komplexe Client-Logik • Sicherheits- und Datenschutzprobleme • Synchronisation schwierig • Abhängigkeit von Client-Verfügbarkeit

Tabelle 1: Vergleich System Architekturen

Als Protokoll zur Kommunikation wird eine REST-Schnittstelle verwendet. Es wurde als Alternative GraphQL betrachtet, allerdings scheint auch hier durch die höhere Komplexität bei der Implementierung im Rahmen des Projekts keinen Mehrwert zu bieten.

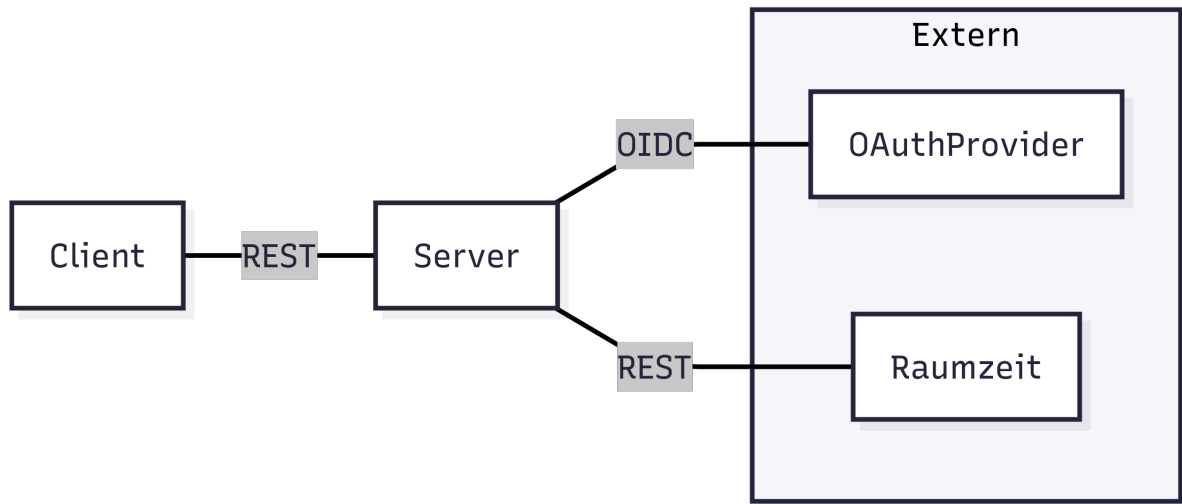


Abbildung 1: Systemarchitektur Lernzeit

Technologieauswahl

Im Rahmen der Analyse wurden für das Backend die Programmiersprachen *Rust*, *Go* und *C#* sowie für das Frontend die Frameworks *Blazor*, *Vue* und *React* evaluiert. Die Tabellen Tabelle 2 und Tabelle 3 listen die jeweiligen Stärken und Schwächen übersichtlich auf.

	Pro	Kontra
Rust	• Höchste Performance • Memory-Safe (Ownership & Borrowing)	• Noch keine Erfahrung im Team • Steile Lernkurve • Saubere Schichtentrennung komplex • Wenige SDKs / externe Bibliotheken
Go	• Erfahrung im Team • Sehr schnelle Entwicklung & einfache Syntax • Einfaches Deployment / kleine Container • Minimalistisch	• Fördert keine saubere Architektur (wenig Guidance) • DI nur durch externe Bibliothek • Weniger Features für komplexe Patterns
C#	• Erfahrung im Team • Ausgereiftes Ecosystem, viele SDKs • Built-in DI & klare Layer-Struktur	• Container größer, Startup langsamer • Framework kann „Magic“ verstecken

Tabelle 2: Vergleich Backend Technologien

	Pro	Kontra
Blazor	• Fullstack mit C# Backend möglich • Starke Typisierung & Compile-Time Checks • Gute Integration in .NET Ecosystem	• Wenig Erfahrung im Team • Weniger reifes Frontend-Ökosystem als JS Frameworks • Lernkurve für WebAssembly-spezifische Konzepte • Weniger Community-Beispiele & Tutorials
Vue	• Erfahrung im Team • Sehr einfache Lernkurve, leicht verständlich • Flexible Komponentenstruktur • Große Community & viele Plugins	• Architektur nicht erzwungen • Bei großen Projekten kann State-Management komplex werden • TypeScript optional, sonst lose Typisierung
React	• Erfahrung im Team • Riesiges Ecosystem & Community • Flexibles Komponentenmodell & Hooks • TypeScript-Integration möglich	• Kein festes Architektur-Muster vorgegeben • Boilerplate bei komplexem State / Redux • Lernkurve bei Hooks & Patterns für Anfänger

Tabelle 3: Vergleich Frontend Technologien

Nach Abwägung der Vor- und Nachteile fiel die Wahl auf die Kombination C# / ASP.NET als Backend-Framework und React als Frontend-Framework.

Diese Entscheidung basiert auf folgenden Kriterien:

- **Architektur:** C# ermöglicht klare Layer-Strukturen und eingebaute Dependency Injection, während React eine flexible Komponentenstruktur bietet, die eine saubere Trennung von Zuständen und Logik unterstützt.
- **Team-Expertise:** Beide Technologien sind im Team bekannt, was eine schnelle Entwicklung und Wissenstransfer erleichtert.
- **Ökosystem & Community:** React und C# bieten jeweils ein umfangreiches Ökosystem, was die Implementierung von komplexen Anforderungen erleichtert. Beide Technologien sind etabliert und haben große Communities.
- **Lern- und Weiterentwicklung:** Die Kombination erlaubt es dem Team, vorhandenes Wissen zu vertiefen und neue Best Practices im Bereich Softwarearchitektur gemeinsam zu erlernen.

Use Cases

1. **Stundenplan importieren:** Die App kann den eigenen Uni-Stundenplan übernehmen. Dafür meldet man sich auf der Hochschul-Plattform an, erstellt einen Freigabe-Link und fügt diesen in Lernzeit ein.
2. **Lerngruppe erstellen:** Um gemeinsame Termine zu planen, erstellt man eine neue Lerngruppe und gibt ihr einen aussagekräftigen Namen - zum Beispiel „Mathe-Lerngruppe“.
3. **Teilnehmer einladen:** Wer eine Gruppe erstellt hat, kann andere über einen Link oder QR-Code weitere Personen einladen. Der QR-Code dient zu einer schnellen und unkomplizierten

Art einer Lerngruppe beizutreten.

4. Freie Zeiten finden und buchen: Die App zeigt Zeiten, in denen alle Gruppenmitglieder frei sind. Diese freien Zeiten kann man als gemeinsame Lernzeit auswählen und optional einen Treffpunkt sowie eine Uhrzeit festlegen.

Muss-/Kann-Kriterien

Mindestanforderungen	• Importieren des eigenen Stundenplanes • Erstellen von Lerngruppe • Hinzufügen von neuen Mitgliedern durch einen Einladungslink oder einen QR-Code • Abgleich von allen Kalenderdaten der Gruppenmitglieder • Visuell überschaubare Darstellung der freie Zeitblöcke
Nice to have	• Möglichkeit zur Abstimmung eines Termins • Anzeige aller frei verfügbarer Räume auf dem Campus • Benachrichtigungen bei neuen Terminvorschlägen oder Änderungen • Integration von Kalender-Apps (Google Calendar, Outlook)